

Práctica No. 4: API de Hilos POSIX

Cristian Echeverry
Ingeniería de Sistemas - Universidad de Antioquia

2026

Datos del estudiante

Nombre	Cristian Echeverry
Correo	cristian.echeverry1@udea.edu.co
Documento	1007566485
Modalidad	Trabajo individual
Video	https://youtu.be/c4qJt1EYYbA

1. Resumen de la solución

Esta práctica implementa el uso de hilos POSIX mediante dos ejercicios. Primero, se paraleliza el cálculo de π por integración numérica usando Pthreads. Segundo, se implementa un generador de secuencia de Fibonacci usando un hilo trabajador y memoria compartida. La solución incluye código fuente, pruebas, scripts de compilación, notebook de análisis y este informe en L^AT_EX.

2. Checklist de cumplimiento

Requisito	Estado	Archivo/Evidencia
Crear versión serial de π	Cumplido	<code>src/pi.c</code>
Crear versión paralela de π con Pthreads	Cumplido	<code>src/pi_p.c</code>
Recibir número de hilos por línea de comandos	Cumplido	<code>./bin/pi_p</code> <code><n><hilos></code>
Particionar el rango de iteraciones	Cumplido	<code>CalcPi()</code> en <code>pi_p.c</code>
Evitar mutex dentro del bucle	Cumplido	Sumas parciales locales
Sincronizar con <code>pthread_join</code>	Cumplido	<code>pi_p.c</code> , <code>fibonacci.c</code>
Crear <code>fibonacci.c</code>	Cumplido	<code>src/fibonacci.c</code>
Pasar arreglo y N al hilo trabajador	Cumplido	Estructura <code>FibArgs</code>
Notebook de análisis	Cumplido	<code>notebooks/analisis.ipynb</code>

Tabla de Speedup y eficiencia	Cumplido	Generada por notebook
Gráfico de Speedup	Cumplido	Generado por notebook
README con informe	Cumplido	README.md
Video	Pendiente	Debe subirse y enlazarse

3. Parte 1: Paralelización del cálculo de π

3.1. Método numérico

El cálculo de π se realiza mediante la integral:

$$\int_0^1 \frac{4}{1+x^2} dx = \pi$$

La aproximación se obtiene dividiendo el intervalo $[0, 1]$ en n rectángulos y aplicando la regla del punto medio.

3.2. Versión serial

La versión serial se implementa en `src/pi.c`. La función principal del cálculo es:

```
double CalcPi(long long n) {
    const double fH = 1.0 / (double)n;
    double fSum = 0.0;

    for (long long i = 0; i < n; i += 1) {
        const double fX = fH * ((double)i + 0.5);
        fSum += f(fX);
    }

    return fH * fSum;
}
```

3.3. Versión paralela

La versión paralela se implementa en `src/pi_p.c`. Se usa una estrategia de paralelismo de datos: el rango total de iteraciones se divide entre T hilos. Cada hilo calcula una suma parcial local y retorna el resultado al hilo principal.

La estructura usada para pasar argumentos a cada hilo es:

```
typedef struct {
    long long start;
    long long end;
```

```
double h;  
} ThreadArgs;
```

Cada hilo ejecuta un subrango $[start, end)$. El hilo principal crea los hilos con `pthread_create` y recolecta las sumas parciales con `pthread_join`. No se usa mutex dentro del bucle, porque cada hilo acumula en una variable local.

3.4. Medición de tiempo

Tanto en `pi.c` como en `pi_p.c` se mide el tiempo con `clock_gettime(CLOCK_MONOTONIC)`. La medición se ubica alrededor de la función de cálculo, con el fin de estimar el tiempo real de ejecución del cálculo de π .

4. Parte 2: Generador de Fibonacci

4.1. Diseño

El archivo `src/fibonacci.c` implementa el cálculo de la secuencia en un hilo trabajador. El hilo principal asigna memoria dinámica para un arreglo compartido y pasa al hilo trabajador una estructura con el puntero al arreglo y el valor N .

```
typedef struct {  
    unsigned long long *values;  
    size_t n;  
} FibArgs;
```

El hilo trabajador escribe los valores de la secuencia en el arreglo compartido. Luego, el hilo principal espera con `pthread_join` y finalmente imprime la secuencia.

4.2. Sincronización

El uso de `pthread_join` es necesario porque impide que `main` acceda al arreglo antes de que el hilo trabajador termine. En este problema no se requiere mutex, porque solo el hilo trabajador escribe en el arreglo y `main` lo lee después de la sincronización.

5. Pruebas

Se incluye el script:

```
./scripts/run_tests.sh
```

Este script compila y prueba:

- `pi_s` con $n = 1,000,000$.
- `pi_p` con $n = 1,000,000$ y 1 hilo.
- `pi_p` con $n = 1,000,000$ y 2 hilos.

- fibonacci con $N = 10$.
- fibonacci con $N = 15$.

6. Notebook de análisis

El notebook `notebooks/analisis.ipynb` automatiza la evaluación solicitada:

1. Ejecuta `./pi_s 2000000000` y obtiene T_s .
2. Ejecuta `./pi_p 2000000000 N` para $N = 1, 2, 4, 8, \dots$ hasta $2 \times$ el número de núcleos.
3. Calcula $Speedup = T_s/T_p$.
4. Calcula $Eficiencia = Speedup/N$.
5. Genera la tabla de resultados y el gráfico de Speedup.
6. Ejecuta `./fibonacci 15`.
7. Ejecuta una versión serial de Fibonacci para $N > 100000$.

7. Problemas y soluciones

Problema	Solución
Contención por variable compartida en el cálculo de π	Se usaron sumas parciales locales y retorno por <code>pthread_join</code> .
División no exacta de iteraciones entre hilos	El residuo se reparte entre los primeros hilos.
Medición de tiempo imprecisa	Se usó <code>clock_gettime(CLOCK_MONOTONIC)</code> alrededor del cálculo.
Riesgo de leer Fibonacci antes de terminar	Se sincroniza con <code>pthread_join</code> .
Overflow para Fibonacci grande	Se documenta que la prueba grande mide desempeño; los valores exactos exceden <code>unsigned long long</code> .

8. Manifiesto de transparencia

crear una versión inicial de los programas en C, generar el notebook de análisis, redactar el README y revisar el cumplimiento del enunciado. La ejecución final de pruebas, validación en el equipo local, creación del video y entrega en Moodle deben ser realizadas por el estudiante.

9. Conclusiones

1. La API de Pthreads permite dividir un trabajo intensivo en CPU entre varios hilos.
2. El cálculo paralelo de π puede reducir el tiempo de ejecución cuando el número de hilos se ajusta al hardware disponible.
3. Evitar mutex dentro del bucle principal mejora el rendimiento al reducir la contención.
4. `pthread_join` es esencial para sincronizar al hilo principal con los hilos trabajadores.
5. El Speedup no crece linealmente de forma indefinida debido al overhead de creación, planificación y sincronización de hilos.
6. La eficiencia disminuye cuando el número de hilos supera la capacidad real de paralelismo del procesador.

10. Referencias

- Remzi H. Arpaci-Dusseau y Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*.
- Capítulo 26: Threads Intro.
- Capítulo 27: Thread API.